

重庆大学计算机专业核心课程《机器学习》

第3章 线性模型

授课教师：郑林江

第3章 线性模型



3.1 基本形式

3.2 线性回归

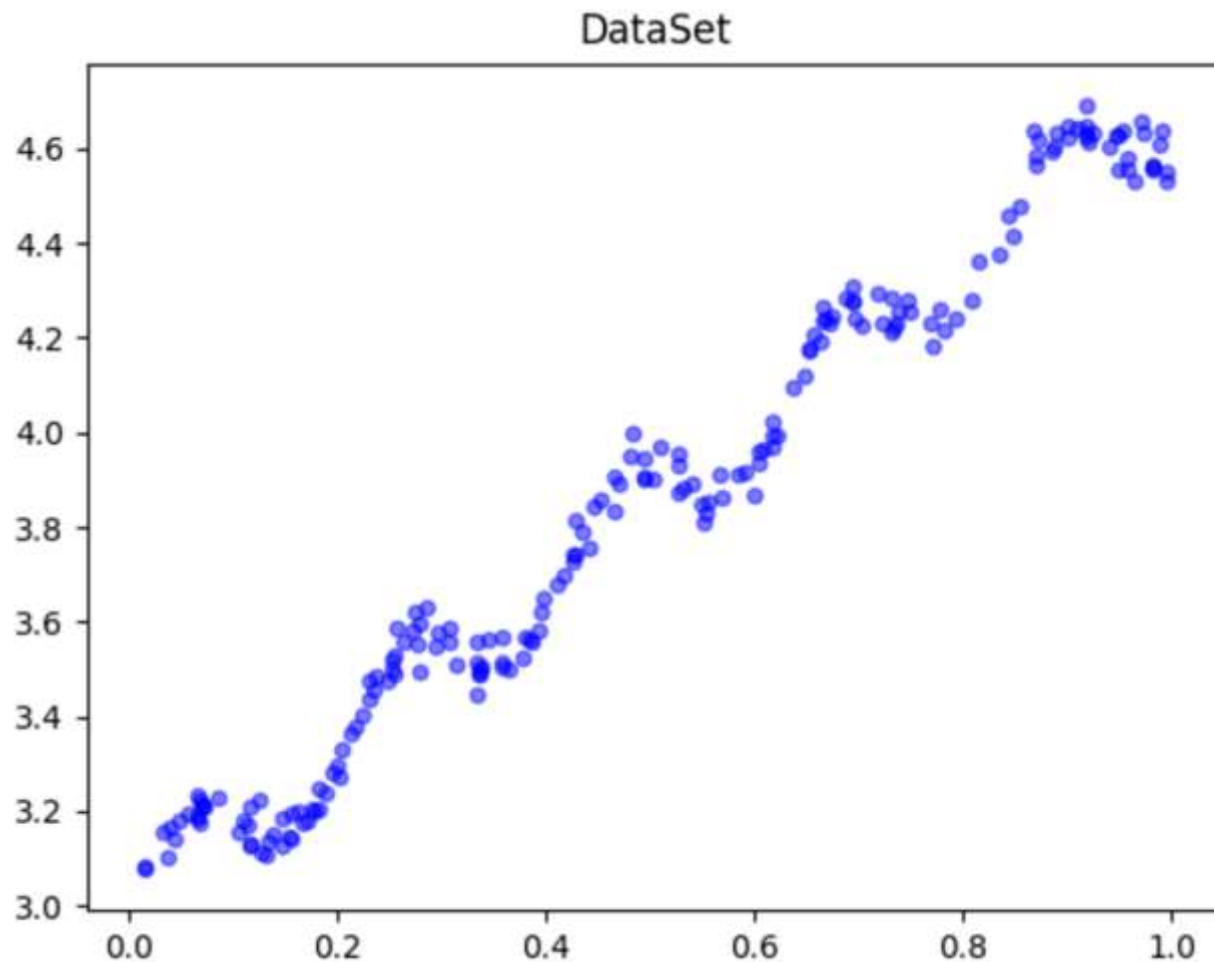
3.3 对数几率回归

3.4 线性判别分析

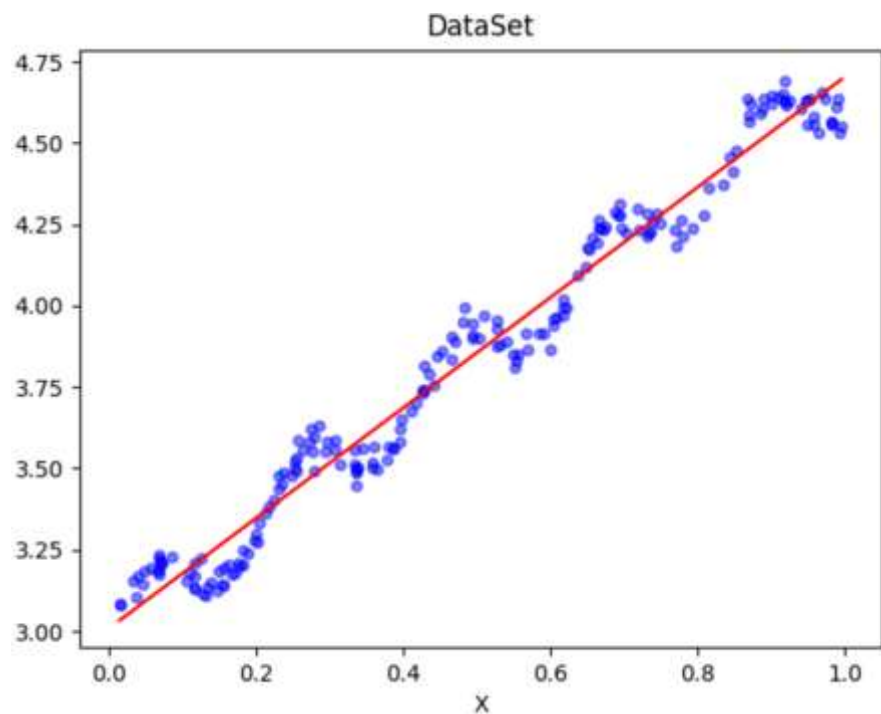
3.5 多分类学习

3.6 类别不平衡问题

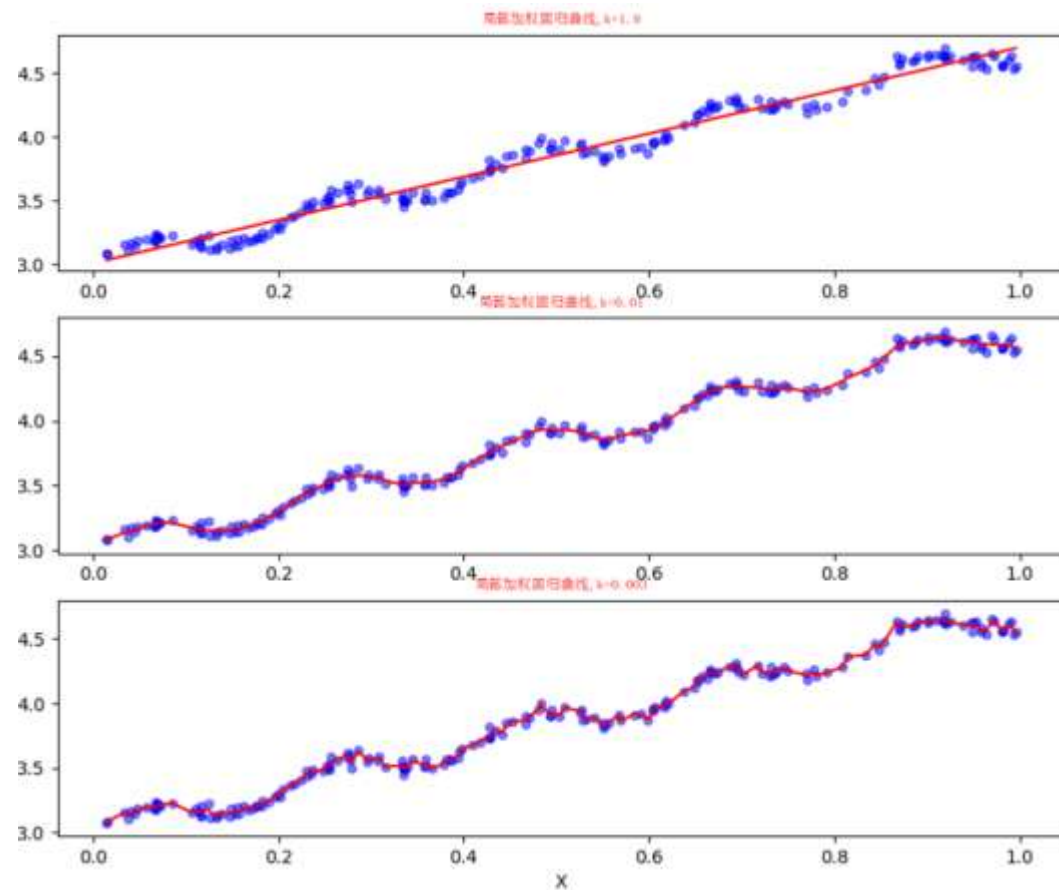
- 回归的目的是预测数值型的目标值。最直接的办法是依据输入写出一个目标值的计算公式。例如某数据集分布如右图：



■ 线性回归



■ 局部加权线性回归



■ 一般形式

- 线性模型一般形式

$$f(\mathbf{x}) = w_1x_1 + w_2x_2 + \dots + w_dx_d + b$$

$\mathbf{x} = (x_1; x_2; \dots; x_d)$ 是由属性描述的示例，其中 x_i 是 $\mathbf{x}$ 在第 i 个属性上的取值

- 向量形式

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$$

其中 $\mathbf{w} = (w_1; w_2; \dots; w_d)$

■ 线性模型优点

- 形式简单、易于建模
- 线性模型引入层级结构或高维映射可得到非线性模型
- 可解释性
- 一个例子
 - 综合考虑色泽、根蒂和敲声来判断西瓜好不好
 - 其中根蒂的系数最大，表明根蒂最要紧；而敲声的系数比色泽大，说明敲声比色泽更重要

$$f_{\text{好瓜}}(\mathbf{x}) = 0.2 \cdot x_{\text{色泽}} + 0.5 \cdot x_{\text{根蒂}} + 0.3 \cdot x_{\text{敲声}} + 1$$

第3章 线性模型

3.1 基本形式



3.2 线性回归

3.3 对数几率回归

3.4 线性判别分析

3.5 多分类学习

3.6 类别不平衡问题

- 给定数据集 $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)\}$
其中: $\mathbf{x}_i = (x_{i1}; x_{i2}; \dots; x_{id})$ $y_i \in \mathbb{R}$
- 线性回归 (linear regression) 目的
 - 学得一个线性模型以尽可能准确地预测实值输出标记
- 离散属性处理
 - 若存在有“序”关系
 - 连续化为连续值 `sklearn.preprocessing.LabelEncoder`
 - 例如: 三值属性高度的取值“高”、“中”、“低”可转化为 $\{1.0, 0.5, 0.0\}$
 - 不存在“序”关系
 - 有k个属性值, 则转换为k维向量 `sklearn.preprocessing.OneHotEncoder`
 - 例如: 属性“瓜类”的取值“西瓜”、“南瓜”、“黄瓜”可转化为 $(0, 0, 1), (0, 1, 0), (1, 0, 0)$

□ 线性回归

□ 单一属性的线性回归目标

$$f(x) = wx_i + b \quad \text{使得} \quad f(x_i) \simeq y_i$$

✓ 如何确定 w 和 b ? 显然, 关键在于如何衡量 $f(x)$ 和 y 之间的差别。

□ 均方误差最小化。

$$\begin{aligned} (w^*, b^*) &= \arg \min_{(w, b)} \sum_{i=1}^m (f(x_i) - y_i)^2 \\ &= \arg \min_{(w, b)} \sum_{i=1}^m (y_i - wx_i - b)^2 \end{aligned}$$

w^*, b^* 表示 w 和 b 的解。

- **均方误差**：有很好的几何意义，对应了常用的**欧式距离**（简称“欧氏距离”）
(Euclidean distance **（为什么欧氏距离重要？）**）。
- **最小二乘法**：基于均方差误差最小化来进行模型求解的方法称为“**最小二乘法**”。
- 在线性回归中，最小二乘法就是试图找到一条直线，使得所有样本到直线上的**欧式距离之和**最小。

□ 线性回归——最小二乘法

□ 求解 w 和 b ，最小化均方误差

$$E_{(w,b)} = \sum_{i=1}^m (y_i - wx_i - b)^2$$

□ 分别对 w 和 b 求导，可得

$$\frac{\partial E_{(w,b)}}{\partial w} = 2 \left(w \sum_{i=1}^m x_i^2 - \sum_{i=1}^m (y_i - b) x_i \right)$$

$$\frac{\partial E_{(w,b)}}{\partial b} = 2 \left(mb - \sum_{i=1}^m (y_i - wx_i) \right)$$

这里 $E_{(w,b)}$ 是关于 w 和 b 的凸函数，当它关于 w 和 b 的导数均为零时，得到 w 和 b 的最优解。

求解 w 和 b 使得 $E_{(w,b)}$ 最小化的过程，称为线性回归模型的最小二乘“参数估计”。

□ 线性回归——最小二乘法

□ 令上式为零，得到 w 和 b 的闭式（closed-form）解

$$w = \frac{\sum_{i=1}^m y_i (x_i - \bar{x})}{\sum_{i=1}^m x_i^2 - \frac{1}{m} \left(\sum_{i=1}^m x_i \right)^2}$$

$$b = \frac{1}{m} \sum_{i=1}^m (y_i - wx_i)$$

其中：

$$\bar{x} = \frac{1}{m} \sum_{i=1}^m x_i$$

□ 线性回归——最小二乘法

□ Sklearn回归

■ 一元线性回归—例子：

预测披萨的价格，

已知价格数据如下表

问：12寸披萨价格是多少？

训练样本	直径（英寸）	价格（美元）
1	6	7
2	8	9
3	10	13
4	14	17.5
5	18	18

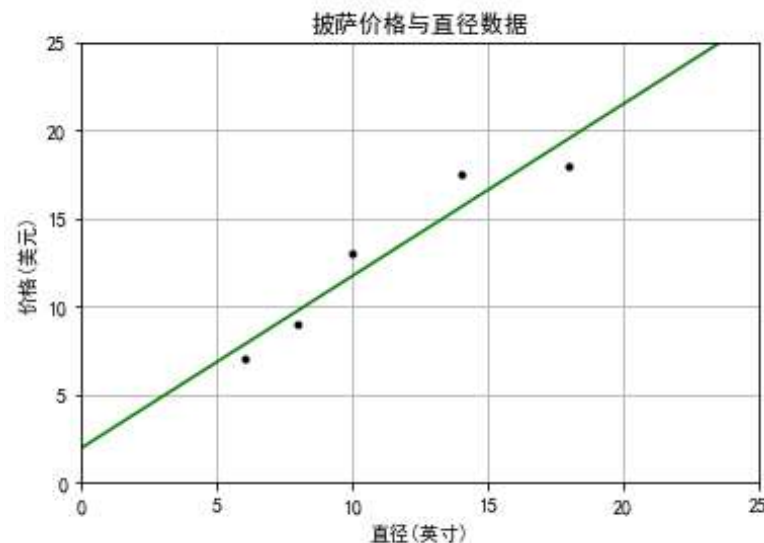
□ 线性回归——最小二乘法

□ Sklearn回归

```
import matplotlib.pyplot as plt
#定义画图函数
def runplt():
    plt.figure()
    plt.rcParams['font.sans-serif'] = 'SimHei'
    plt.rcParams['axes.unicode_minus'] = False
    plt.title('披萨价格与直径数据')
    plt.xlabel('直径(英寸)')
    plt.ylabel('价格(美元)')
    plt.axis([0,25,0,25])
    plt.grid(True)
    return plt
```

```
#训练集数据
X = [[6], [8], [10], [14], [18]]
y = [[7], [9], [13], [17.5], [18]]

#导入一元线性回归函数: $y = a + bx$ 
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X,y) #训练集数据放入模型中
print('预测一张12英寸披萨价格: %.2f' % model.predict([[12]]))
pplt = runplt()
#pplt.rcParams['font.family']='SimHei'
X2 = [[0],[10],[14],[25]]
y2 = model.predict(X2) #预测数据
pplt.plot(X,y,'k.')
pplt.plot(X2,y2,'g-')
pplt.show()
```



■ 预测一张12英寸的披萨的价格: **\$ 13.68**

- 更一般的情形，给定数据集

$$D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)\}$$

$$\mathbf{x}_i = (x_{i1}; x_{i2}; \dots; x_{id}) \quad y_i \in \mathbb{R}$$

- 多元线性回归目标

$$f(\mathbf{x}_i) = \mathbf{w}^T \mathbf{x}_i + b \quad \text{使得 } f(\mathbf{x}_i) \simeq y_i$$

□ 线性回归——多元线性回归

- 类似地，可利用最小二乘法来 $\mathbf{w}$ 和 b 进行估计。
- 为便于讨论，把 $\mathbf{w}$ 和 b 吸收入向量形式 $\hat{\mathbf{w}} = (\mathbf{w}; b)$ 。
- 相应地，把数据集 D 表示为一个 $m \times (d+1)$ 的矩阵 $\mathbf{X}$

$$\text{矩阵 } \mathbf{X} = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1d} & 1 \\ x_{21} & x_{22} & \cdots & x_{2d} & 1 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ x_{m1} & x_{m2} & \cdots & x_{md} & 1 \end{pmatrix} = \begin{pmatrix} \mathbf{x}_1^T & 1 \\ \mathbf{x}_2^T & 1 \\ \vdots & \vdots \\ \mathbf{x}_m^T & 1 \end{pmatrix}$$

其中：

- ✓ 每行对应于一个示例
- ✓ 前 d 个元素对应于示例的 d 个属性值
- ✓ 最后一个元素恒置为 1

- 再把标记写成向量形式： $\mathbf{y} = (y_1; y_2; \cdots; y_m)$

□ 线性回归——多元线性回归

- 采用最小二乘法 (least square method)

$$\hat{\mathbf{w}}^* = \arg \min_{\hat{\mathbf{w}}} (\mathbf{y} - \mathbf{X}\hat{\mathbf{w}})^T (\mathbf{y} - \mathbf{X}\hat{\mathbf{w}})$$

令 $E_{\hat{\mathbf{w}}} = (\mathbf{y} - \mathbf{X}\hat{\mathbf{w}})^T (\mathbf{y} - \mathbf{X}\hat{\mathbf{w}})$, 对 $\hat{\mathbf{w}}$ 求导得到

$$\frac{\partial E_{\hat{\mathbf{w}}}}{\partial \hat{\mathbf{w}}} = 2\mathbf{X}^T (\mathbf{X}\hat{\mathbf{w}} - \mathbf{y})$$

令上式为零可得 $\hat{\mathbf{w}}$ 最优解的闭式解。

由于涉及矩阵逆的计算，比单变量复杂些

□ 若 $X^T X$ 是满秩矩阵或正定矩阵，令 $\hat{w} = \frac{\partial E_{\hat{w}}}{\partial \hat{w}}$

$$\hat{w}^* = (X^T X)^{-1} X^T y$$

其中： $(X^T X)^{-1}$ 是 $X^T X$ 的逆矩阵。令 $\hat{x}_i = (x_i, 1)$ ，则最终学得的多元线性回归模型为

$$f(\hat{x}_i) = \hat{x}_i^T (X^T X)^{-1} X^T y$$

■ 若 $X^T X$ 不是满秩矩阵，可解出多个 $\hat{w}$ ，都能使得均方误差最小化，选择哪个作为输出？

- 根据归纳偏好选择解（参见1.4节）
- 常见的有正则化（参见6.4节，11.4节）

□ 多元线性回归

- 现实中披萨价格的影响因素不止直径一个, 引入了辅料的因素:

训练集数据:

训练样本	直径 (英寸)	辅料种类	价格 (美元)
1	8	11	9.7759
2	9	8.5	10.7522
3	11	15	12.7048
4	16	18	17.5863
5	12	11	13.6811

测试集数据:

测试样本	直径 (英寸)	辅料种类	价格 (美元)
1	8	2	11
2	9	0	8.5
3	11	2	15
4	16	2	18
5	12	0	11

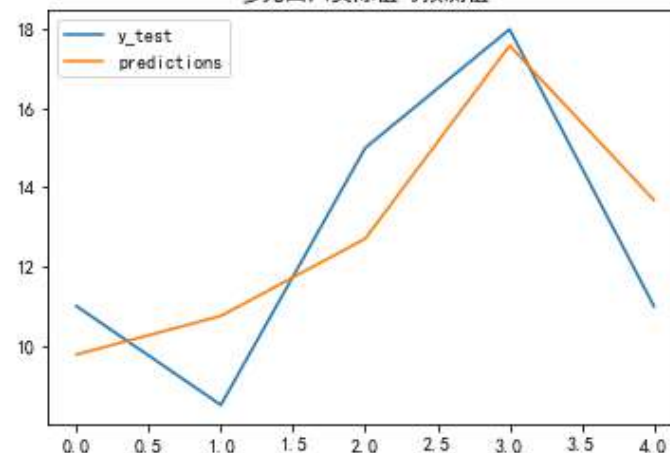
#训练集, 多元线性回归模型训练

```
X = [[8, 11], [9, 8.5], [11, 15], [16, 18], [12, 11]]  
y = [[9.7759], [10.7522], [12.7048], [17.5863], [13.6811]]  
model = LinearRegression()  
model.fit(X,y)
```

#测试集, 及预测

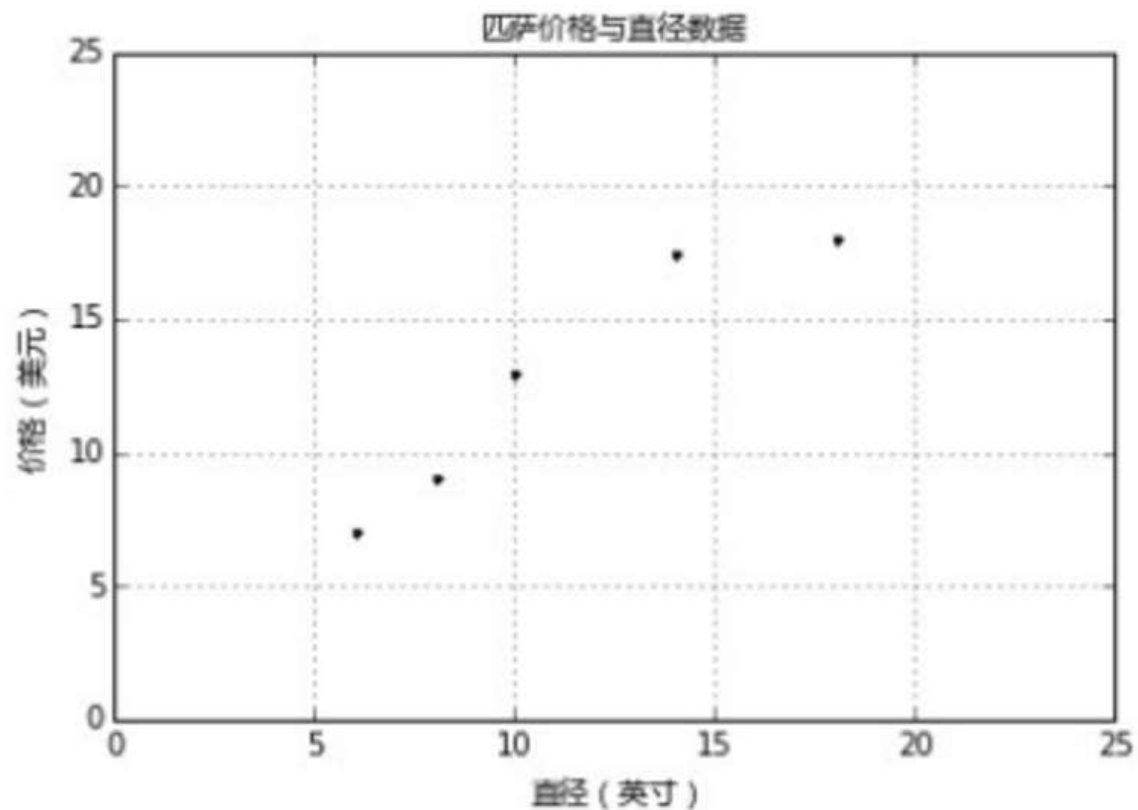
```
X_test = [[8, 2], [9, 0], [11, 2], [16, 2], [12, 0]]  
y_test = [[11], [8.5], [15], [18], [11]]  
predictions = model.predict(X_test)
```

多元回归实际值与预测值



□ 多元线性回归

- 从训练集原始数据的散点图来看：其实也有可能是一个曲线模型，试下二次回归的效果

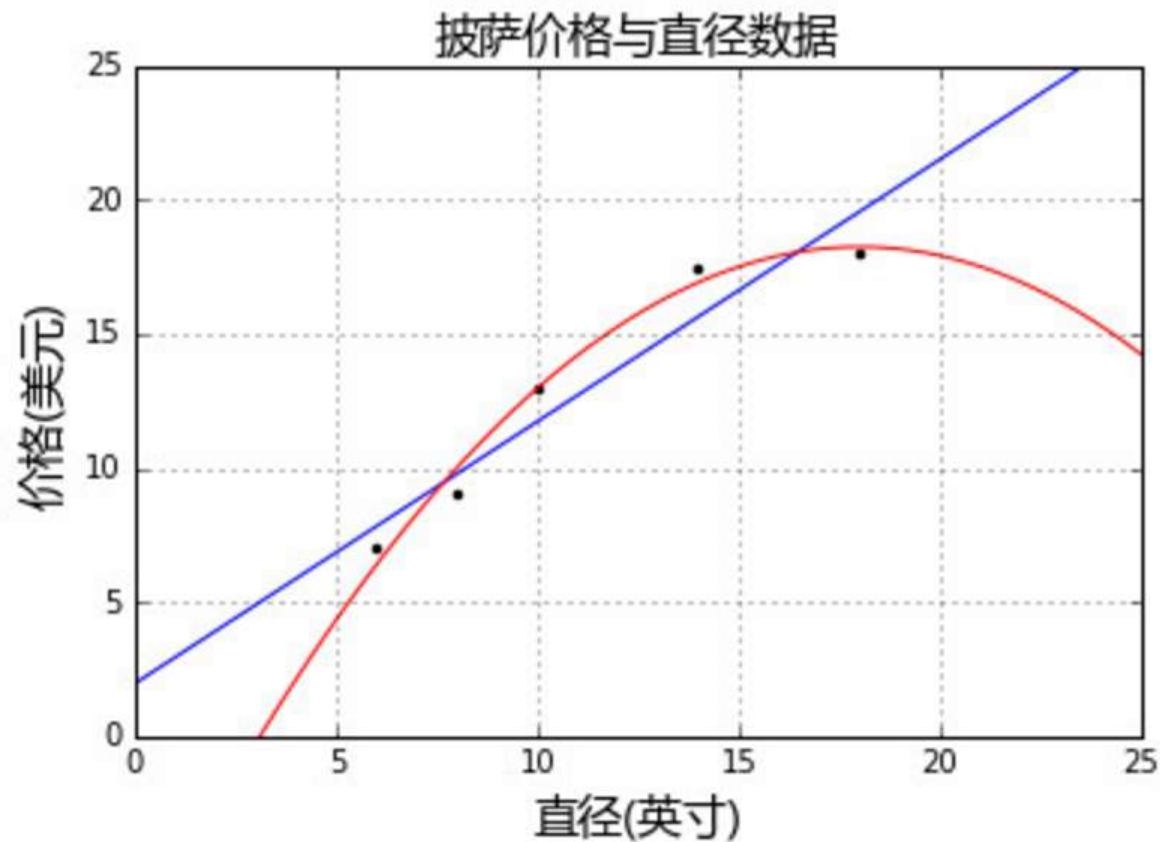


- 用只有直径一个因素的训练集数据,二次回归 (Quadratic Regression)
- $y = \alpha + \beta_1 x + \beta_2 x^2$, 通过模型第三项 (二次项) 来实现曲线关系, PolynomialFeatures 转换器可以用来解决这个问题。

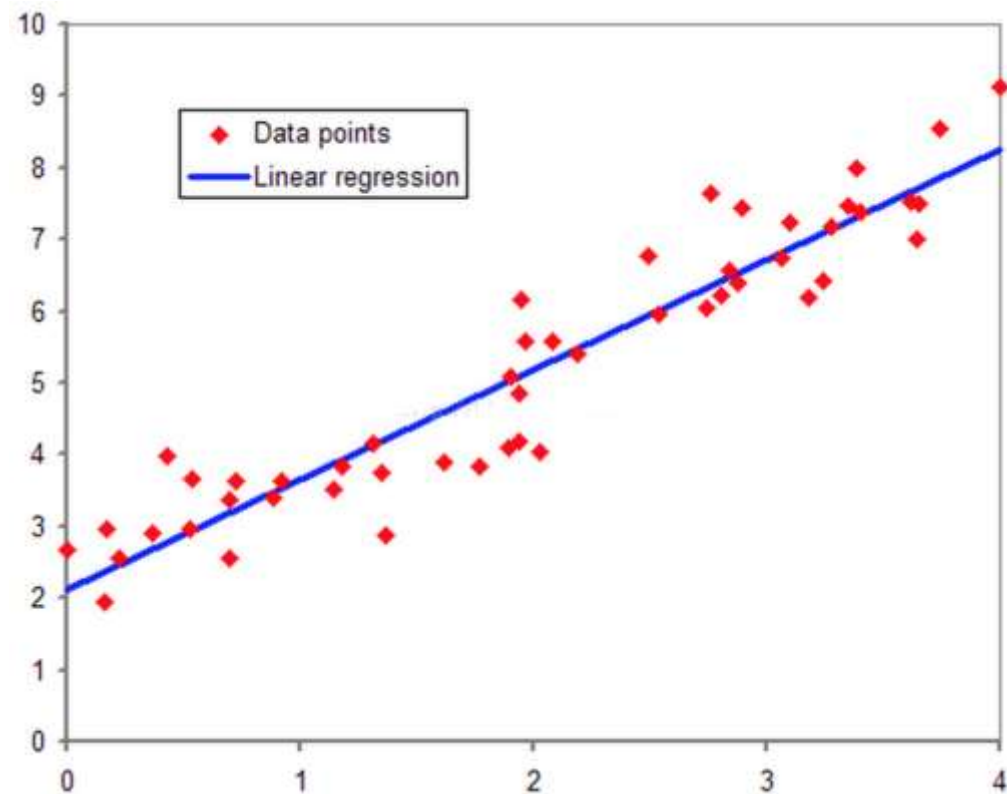
```
#构造第三项
quadratic_fearurizer = PolynomialFeatures(degree=2)
X_train_quadratic = quadratic_fearurizer.fit_transform(X_train)
X_test_quadratic = quadratic_fearurizer.transform(X_test)
regressor_quadratic = LinearRegression()
regressor_quadratic.fit(X_train_quadratic, y_train)
xx_quadratic = quadratic_fearurizer.transform(xx.reshape(xx.shape[0],1))
plt.plot(xx,regressor_quadratic.predict(xx_quadratic),'r-')
plt.show()
```

□ 多元线性回归

- 看起来二次回归效果比线性回归好。那么三次回归，四次回归效果会不会更好呢？
- 继续尝试发现不是。



- 线性回归对于多维空间中存在的样本点，用特征的线性组合去拟合空间中点的分布和轨迹。
- 线性回归用于对连续值结果进行预测。



```
sklearn.linear_model.LinearRegression
```

```
sklearn.linear_model.Lasso
```

```
sklearn.linear_model.Ridge
```


□ 线性回归——对数线性回归

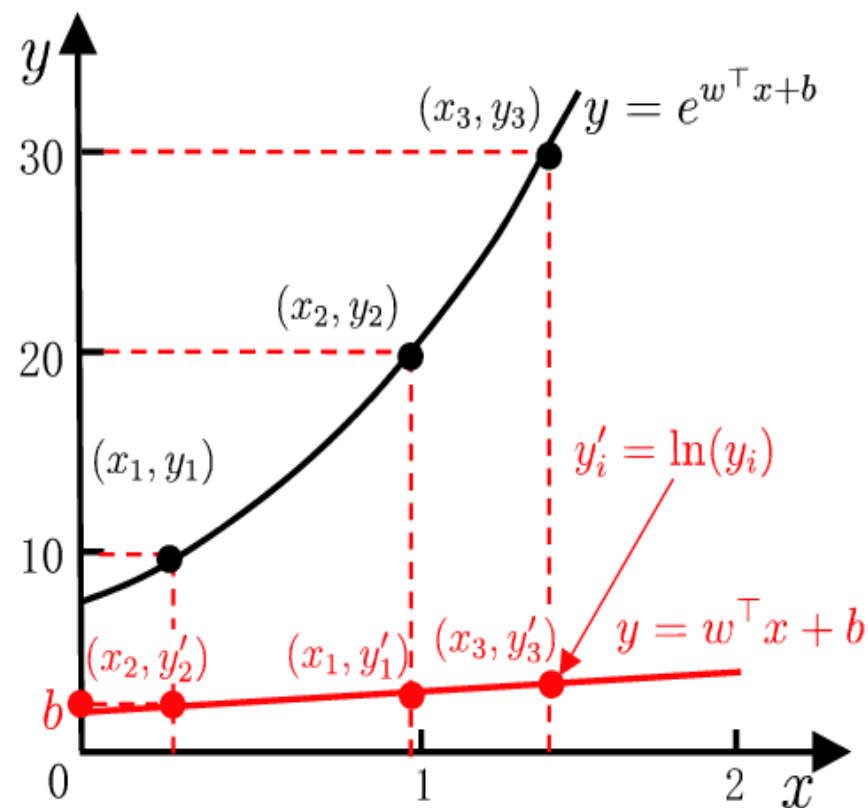
- 可否令模型 $y = w^T x + b$ 的预测值逼近 y 的衍生物？
- 将输出标记的**对数**为线性模型逼近的目标（输出在指数尺度上变化）。

线性回归 $y = w^T x + b$



对数线性回归 $\ln y = w^T x + b$

- 实际上是试图让 $e^{w^T x + b}$ 逼近 y 。



数函数起到了将线性回归模型的预测值与真实标记联系起来的作用。

- 一般形式，考虑单调可微函数 $g(\cdot)$ ，令：

$$y = g^{-1}(\boldsymbol{w}^T \boldsymbol{x} + b)$$

- 这样得到的模型称为“广义线性模型”。
- 其中： $g(\cdot)$ 称为联系函数 (link function)
 - 单调可微函数 (单调才可求逆)

- 对数线性回归是广义线性模型在 $g(\cdot) = \ln(\cdot)$ 的特例。

第3章 线性模型

3.1 基本形式

3.2 线性回归

➔ 3.3 对数几率回归

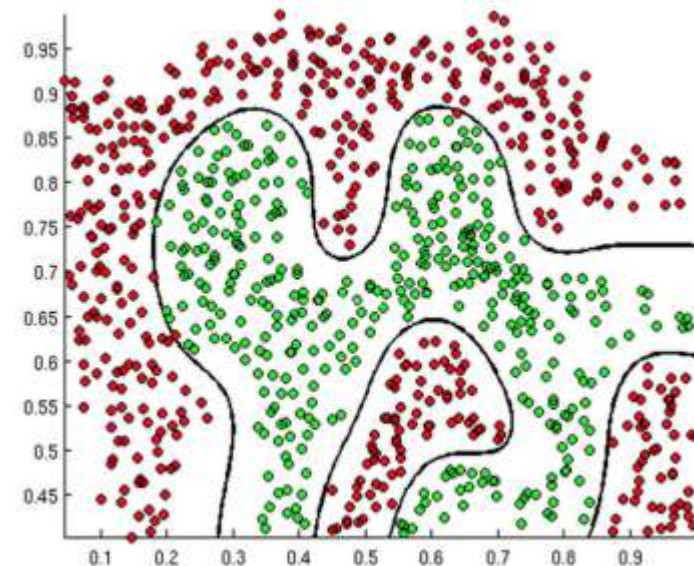
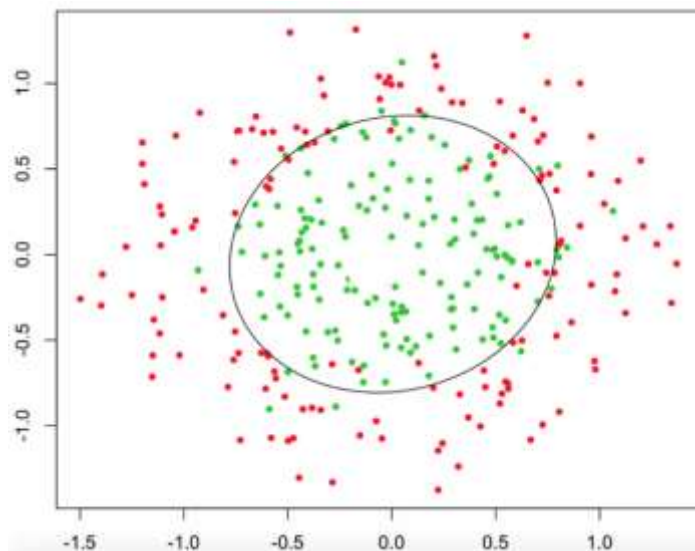
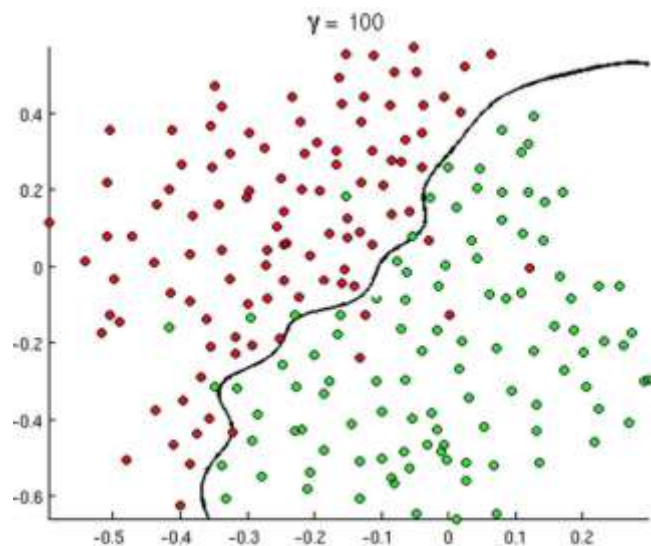
3.4 线性判别分析

3.5 多分类学习

3.6 类别不平衡问题

□ 对数几率回归（逻辑回归）

- 线性回归是针对回归任务，若要做分类任务，如何处理？



□ 对数几率回归（逻辑回归）

- 考虑二分类任务。预测值与输出标记

$$y \in \{0, 1\} \quad \text{二分类任务}$$

$$z = \boldsymbol{w}^T \boldsymbol{x} + b \quad \text{线性模型结果}$$

- 寻找函数将分类标记与线性回归模型输出联系起来。

- 我们需要将实值 z 转换为0/1。最理想的函数——单位阶跃函数

$$y = \begin{cases} 0, & z < 0; \\ 0.5, & z = 0; \\ 1, & z > 0, \end{cases}$$

□ 对数几率回归（逻辑回归）

- 预测值大于零就判为正例，小于零就判为反例，预测值为临界值零则可任意判别

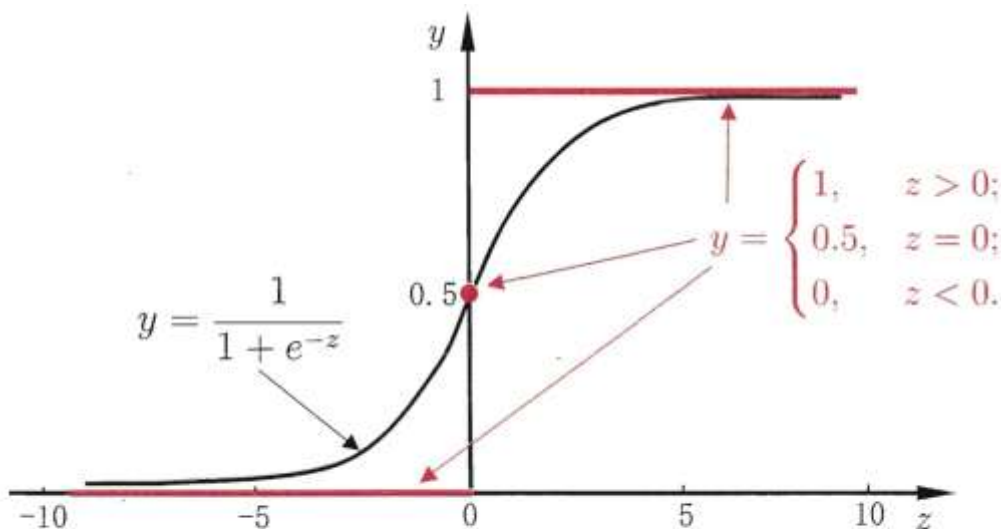


图 3.2 单位阶跃函数与对数几率函数

- ✓ **单位阶跃函数缺点：不连续**
- ✓ 希望能找到一定程度近似单位阶跃函数的替代函数
- ✓ **替代函数——对数几率函数** (logistic function) (**sigmoid**函数，形状类似S)

$$y = \frac{1}{1 + e^{-z}}$$

- 对数几率函数将y值转化为接近0或1的。
- 单调可微、任意阶可导

□ 对数几率回归（逻辑回归）

■ 运用对数几率函数

$$y = \frac{1}{1 + e^{-z}} \quad \text{变为} \quad y = \frac{1}{1 + e^{-(w^T x + b)}}$$

$$\text{进一步可变为: } \ln \frac{y}{1-y} = w^T x + b$$

■ 对数几率回归优点

- ✓ 无需事先假设数据分布
- ✓ 可得到“类别”的近似概率预测
- ✓ 可直接应用现有数值优化算法求取最优解

■ 对数几率（log odds）

✓ 样本作为正例的相对可能性的对数， y 是正例可能性， $1-y$ 是反例可能性。两者比值：

$$\text{几率: } \frac{y}{1-y}$$

✓ 反映了 x 作为正例的相对可能性。

✓ 对几率取对数，则得到对数几率。

$$\ln \frac{y}{1-y} \quad \text{对数几率}$$

□ 对数几率回归（逻辑回归）——极大似然

- 如何确定 $\mathbf{w}$ 和 b ?
- 将 y 视为类后验概率 $p(y = 1 | \mathbf{x})$ ，对数几率可重写为：

$$\ln \frac{p(y = 1 | \mathbf{x})}{p(y = 0 | \mathbf{x})} = \mathbf{w}^T \mathbf{x} + b$$

- 显然有（两者相加为1）

$$p(y = 1 | \mathbf{x}) = \frac{e^{\mathbf{w}^T \mathbf{x} + b}}{1 + e^{\mathbf{w}^T \mathbf{x} + b}}$$

$$p(y = 0 | \mathbf{x}) = \frac{1}{1 + e^{\mathbf{w}^T \mathbf{x} + b}}$$

□ 对数几率回归（逻辑回归）——极大似然

■ 极大似然法（maximum likelihood）

- 通过极大似然法来估计 w 和 b
- 给定数据集： $\{(\mathbf{x}_i, y_i)\}_{i=1}^m$

- 对数回归模型最大化“对数似然”

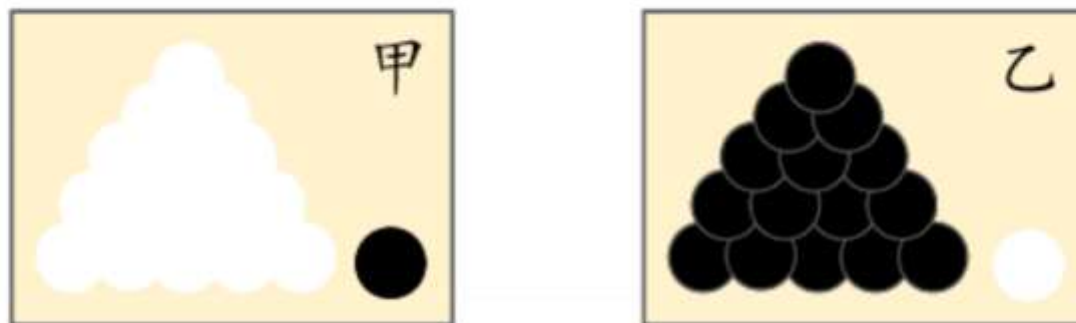
$$\ell(\mathbf{w}, b) = \sum_{i=1}^m \ln p(y_i | \mathbf{x}_i; \mathbf{w}_i, b)$$

- 即令每个样本属于其真实标记的概率越大越好。
 - 最大化对数似然

□ 对数几率回归（逻辑回归）——极大似然

■ 极大似然法（maximum likelihood）

◆ 最大似然原理



- 例：有两个外形完全相同的箱子，甲箱中有99只白球，1只黑球；乙箱中有99只黑球，1只白球。一次试验取出一球，结果取出的是黑球。
- 问：黑球从哪个箱子中取出？
- 人们的第一印象就是：“此黑球最像是从乙箱中取出的”，这个推断符合人们的经验事实。“最像”就是“最大似然”之意，这种想法常称为“最大似然原理”（maximum-likelihood）。

□ 对数几率回归（逻辑回归）——极大似然

■ 极大似然法（maximum likelihood）

✓ 转化为**最小化负**对数似然函数求解

• 令 $\beta = (w; b)$, $\hat{x} = (x; 1)$ 则 $w^T x + b$ 简写为 $\beta^T \hat{x}$

• 再令: $p_1(\hat{x}_i; \beta) = p(y = 1 | \hat{x}; \beta)$

$$p_0(\hat{x}_i; \beta) = p(y = 0 | \hat{x}; \beta) = 1 - p_1(\hat{x}_i; \beta)$$

• 则似然项可重写为

$$p(y_i | x_i; w_i, b) = y_i p_1(\hat{x}_i; \beta) + (1 - y_i) p_0(\hat{x}_i; \beta)$$

✓ 故**等价形式为要最小化(高阶可导连续凸函数，可用梯度下降、牛顿法)**

$$\ell(\beta) = \sum_{i=1}^m \left(-y_i \beta^T \hat{x}_i + \ln \left(1 + e^{\beta^T \hat{x}_i} \right) \right)$$

■ 极大似然法（maximum likelihood）

- ✓ 根据凸优化理论，经典的数值优化方法如梯度下降、牛顿法等都可求得其最优解。得到： $\beta^* = \arg \min_{\beta} \ell(\beta)$

- ✓ 以牛顿法为例：第t+1轮迭代解的更新公式

$$\beta^{t+1} = \beta^t - \left(\frac{\partial^2 \ell(\beta)}{\partial \beta \partial \beta^T} \right)^{-1} \frac{\partial \ell(\beta)}{\partial \beta}$$

其中关于 β 的一阶、二阶导数分别为

$$\frac{\partial \ell(\beta)}{\partial \beta} = - \sum_{i=1}^m \hat{\mathbf{x}}_i (y_i - p_1(\hat{\mathbf{x}}_i; \beta)) \quad \frac{\partial^2 \ell(\beta)}{\partial \beta \partial \beta^T} = \sum_{i=1}^m \hat{\mathbf{x}}_i \hat{\mathbf{x}}_i^T p_1(\hat{\mathbf{x}}_i; \beta) (1 - p_1(\hat{\mathbf{x}}_i; \beta))$$

高阶可导连续凸函数，梯度下降法/牛顿法 [Boyd and Vandenberghe, 2004]

□ 对数几率回归（逻辑回归）——极大似然

- **鸢尾花数据集**是数据挖掘常用一个数据集；鸢尾花数据集采集的是鸢尾花的测量数据以及其所属的类别。测量数据包括：萼片长度、萼片宽度、花瓣长度、花瓣宽度。
- 类别共分为三类：Iris Setosa, Iris Versicolour, Iris Virginica。该数据集可用于多分类问题。

萼片长度	萼片宽度	花瓣长度	花瓣宽度	类别
5.1	3.5	1.4	0.2	Iris-setosa
4.9	3	1.4	0.2	Iris-setosa
4.7	3.2	1.3	0.2	Iris-setosa
4.6	3.1	1.5	0.2	Iris-setosa
5	3.6	1.4	0.2	Iris-setosa
5.4	3.9	1.7	0.4	Iris-setosa
4.6	3.4	1.4	0.3	Iris-setosa
5	3.4	1.5	0.2	Iris-setosa
4.4	2.9	1.4	0.2	Iris-setosa
4.9	3.1	1.5	0.1	Iris-setosa
5.4	3.7	1.5	0.2	Iris-setosa
4.8	3.4	1.6	0.2	Iris-setosa
		1.4	0.1	Iris-setosa
		1.1	0.1	Iris-setosa
		1.2	0.2	Iris-setosa



□ 对数几率回归（逻辑回归）——极大似然

■ 加载数据集

```
>>> from sklearn.datasets import load_iris
>>> iris = load_iris()
>>> print(iris.data.shape)
(150, 4)
>>> print(iris.target.shape)
(150, )
>>> list(iris.target_names)
['setosa', 'versicolor', 'virginica']
```


□ 对数几率回归（逻辑回归）——极大似然

■ Sklearn逻辑回归

```
from sklearn import datasets
from sklearn.cross_validation import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report
iris = datasets.load_iris()
X_train = iris.data
y_train = iris.target
X_train,X_test,y_train,y_test = train_test_split(X_train,y_train,\
        test_size=0.25,random_state=0,stratify=y_train)
regr = LogisticRegression()
regr.fit(X_train,y_train)
```

```
print('Coefficients :%s,intercept %s'%(regr.coef_,regr.intercept_))
print('Score: %.2f'%regr.score(X_test,y_test))
print(classification_report(y_test,regr.predict(X_test)))
```

Coefficients :[[0.39310895 1.35470406 -2.12308303 -0.96477916]
[0.22462128 -1.34888898 0.60067997 -1.24122398]
[-1.50918214 -1.29436177 2.14150484 2.2961458]],intercept [0.24122458 1.13775782 -1.09418724]
Score: 0.97

	precision	recall	f1-score	support
0	1.00	1.00	1.00	13
1	1.00	0.92	0.96	13
2	0.92	1.00	0.96	12
avg / total	0.98	0.97	0.97	38

第3章 线性模型

3.1 基本形式

3.2 线性回归

3.3 对数几率回归

➡ 3.4 线性判别分析

3.5 多分类学习

3.6 类别不平衡问题

□ 线性判别分析LDA

- LDA是一种经典的线性学习方法。在二分类问题上因为最早由 [Fisher, 1936] 提出，亦称 “Fisher判别分析”。
- 线性判别分析LDA思想：
 - ✓ 给定**训练样例集**，设法将样例投影到一条直线上，使得同类样例的投影点尽可能接近、异类样例的投影点尽可能远离。
 - ✓ 对**新样本进行分类**时，将其投影到同样的这条直线上，再根据投影点的位置来确定新样本的类别。

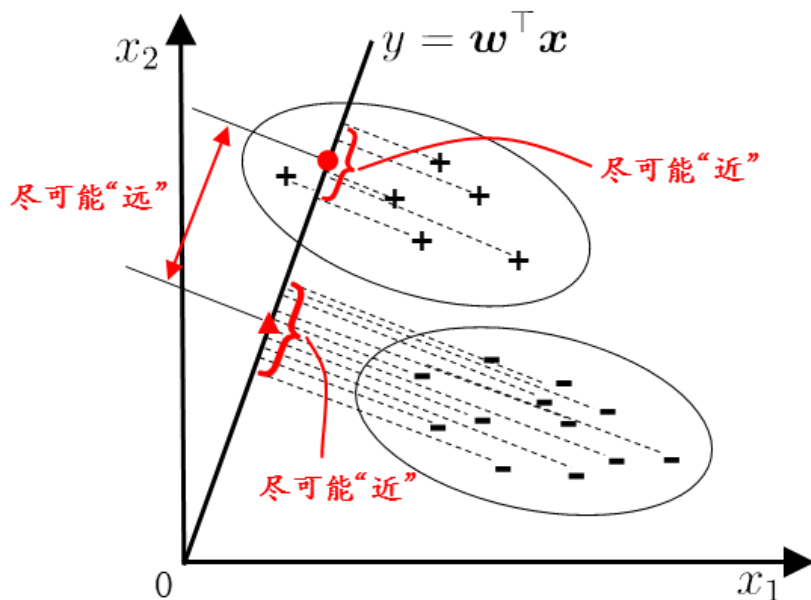
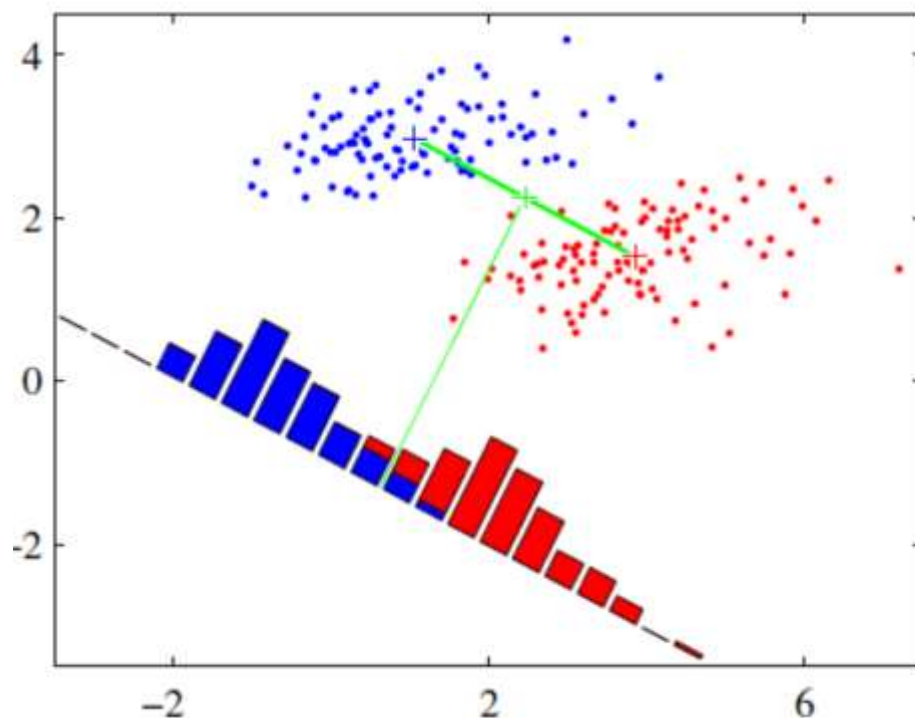


图 3.3 LDA 的二维示意图。“+”、“-”分别代表正例和反例，椭圆表示数据簇的外轮廓，虚线表示投影，红色实心圆和实心三角形分别表示两类样本投影后的中心点。

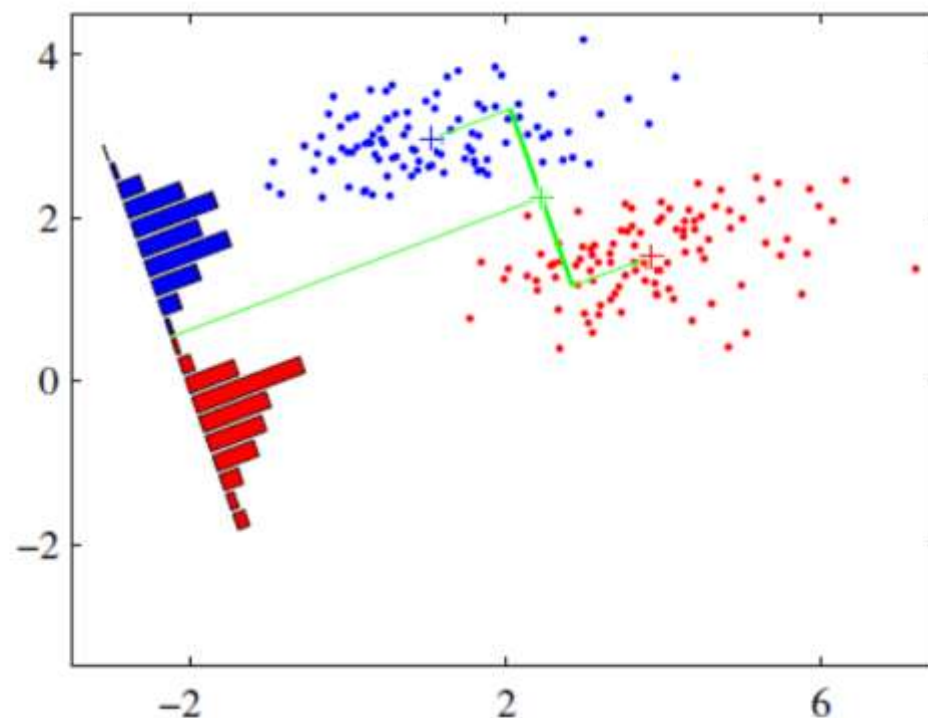
- ✓ LDA也可被视为一种**监督降维技术**
- ✓ PCA是**无监督降维**

□ 线性判别分析LDA

■ LDA是一种经典的线性学习方法。



投影方式A



投影方式B

A和B，哪个更好？

■ LDA思想

- ✓ 欲使同类样例的投影点尽可能接近，可以让同类样例投影点的协方差尽可能小。
- ✓ 欲使异类样例的投影点尽可能远离，可以让类中心之间的距离尽可能大。
- ✓ 给定数据集 $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^m$, $y_i \in \{0, 1\}$ ，一些变量：
 - ◆ 第i类示例的集合 X_i
 - ◆ 第i类示例的均值向量 μ_i
 - ◆ 第i类示例的协方差矩阵 Σ_i
 - ◆ 则两类样本的中心在直线上的投影： $w^T \mu_0$ 和 $w^T \mu_1$
 - ◆ 若将所有杨蓓投影到直线上，两类样本的协方差： $w^T \Sigma_0 w$ 和 $w^T \Sigma_1 w$

□ 线性判别分析LDA——二分类任务

分子: 类中心之间的距离
能大
分母: 同类样例投影点的
差, 尽可能小

■ LDA思想

- ✓ 欲使同类样例的投影点尽可能接近, 可以让同类样例投影点的**协方差**尽可能小, 即

$$w^T \Sigma_0 w + w^T \Sigma_1 w \text{ 尽可能小。}$$

- ✓ 而欲使异类样例的投影点尽可能远离, 可以让**类中心之间的距离**尽可能大, 即

$$\|w^T \mu_0 - w^T \mu_1\|_2^2 \text{ 尽可能大。}$$

- ✓ 同时考虑二者, 则可得到欲**最大化**的目标:

$$\begin{aligned} J &= \frac{\|w^T \mu_0 - w^T \mu_1\|_2^2}{w^T \Sigma_0 w + w^T \Sigma_1 w} \\ &= \frac{w^T (\mu_0 - \mu_1) (\mu_0 - \mu_1)^T w}{w^T (\Sigma_0 + \Sigma_1) w} \end{aligned}$$

分子: 类中心之间的距离
，尽可能大

分母: 同类样例投影点的
协方差, 尽可能小

□ 线性判别分析LDA——二分类任务

■ 类内散度矩阵

$$\begin{aligned} S_w &= \Sigma_0 + \Sigma_1 \\ &= \sum_{x \in X_0} (x - \mu_0)(x - \mu_0)^T + \sum_{x \in X_1} (x - \mu_1)(x - \mu_1)^T \end{aligned}$$

■ 类间散度矩阵

$$S_b = (\mu_0 - \mu_1)(\mu_0 - \mu_1)^T$$

■ LDA欲最大化目标重写为:

$$J = \frac{w^T S_b w}{w^T S_w w}$$

分子: 类间散度
分母: 类内散度

✓ 得到 S_b 和 S_w 的广义瑞利商 (generalized Rayleigh quotient)

□ 线性判别分析LDA——二分类任务

■ 如何确定 w 勒？

$$J = \frac{w^T S_b w}{w^T S_w w}$$

- ✓ 分子和分母都是关于 w 的二次项，因此其解与 w 的长度无关，只与其方向有关。
- ✓ 不是一般性，令 $w^T S_w w = 1$ ，最大化广义瑞利商等价形式为：


$$\begin{aligned} \min_w \quad & -w^T S_b w \\ \text{s.t.} \quad & w^T S_w w = 1 \end{aligned}$$

□ 运用拉格朗日乘子法（带约束优化问题求解）

$$S_b w = \lambda S_w w \quad \text{其中：}\lambda\text{是拉格朗日乘子。}$$

□ 线性判别分析LDA——二分类任务

■ 如何确定 w 勒？

□ 注意到 $S_b w$ 的方向恒为 $\mu_0 - \mu_1$ ，不妨令：

$$S_b w = \lambda (\mu_0 - \mu_1)$$


同向向量

□ 得到结果

$$w = S_w^{-1} (\mu_0 - \mu_1)$$

□ 如何求解

考虑到数值解的稳定性，实践中通常对 S_w 进行奇异值分解。即： $S_w = U \Sigma V^T$

然后再由 $S_w^{-1} = V \Sigma^{-1} U^T$ 得到 S_w^{-1} 。

□ LDA的贝叶斯决策论解释

✓ 两类数据同先验、满足高斯分布且协方差相等时，LDA达到最优分类。

□ 线性判别分析LDA——多分类任务

■ LDA推广到多分类任务。

□ 假定存在 N 个类，且第 i 类示例数为 m_i 。我们先定义“全局散度矩阵”。

$$\begin{aligned} \mathbf{S}_t &= \mathbf{S}_b + \mathbf{S}_w \\ &= \sum_{i=1}^m (\mathbf{x}_i - \boldsymbol{\mu})(\mathbf{x}_i - \boldsymbol{\mu})^T \end{aligned} \quad \textcolor{red}{u} \text{是所有示例的均值向量。}$$

□ 将类内散度矩阵 $\mathbf{S}_w$ 重定义为每个类别的散度矩阵之和。

$$\mathbf{S}_w = \sum_{i=1}^N \mathbf{S}_{w_i}$$

$$\text{其中: } \mathbf{S}_{w_i} = \sum_{\mathbf{x} \in X_i} (\mathbf{x} - \boldsymbol{\mu}_i)(\mathbf{x} - \boldsymbol{\mu}_i)^T$$

□ 线性判别分析LDA——多分类任务

■ LDA推广到多分类任务。

□ 求解得到: $S_b = S_t - S_w$

$$= \sum_{i=1}^N m_i (\mu_i - \mu) (\mu_i - \mu)^T$$

□ 显然，多分类LDA可以有多种实现方法：使用 S_b, S_g, S_t 三者中的任何两个即可。常见的一种实现是采用优化目标：

$$\max_{\mathbf{W}} \frac{\text{tr}(\mathbf{W}^T \mathbf{S}_b \mathbf{W})}{\text{tr}(\mathbf{W}^T \mathbf{S}_w \mathbf{W})}$$

其中： $\mathbf{W} \in \mathbb{R}^{d \times (N-1)}$ 表示矩阵的迹。

□ 线性判别分析LDA——多分类任务

■ LDA推广到多分类任务。

□ 优化目标可通过如下广义特征值问题求解：

$$\mathbf{S}_b \mathbf{W} = \lambda \mathbf{S}_w \mathbf{W}$$

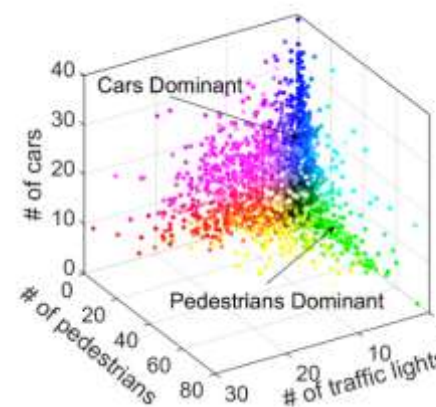
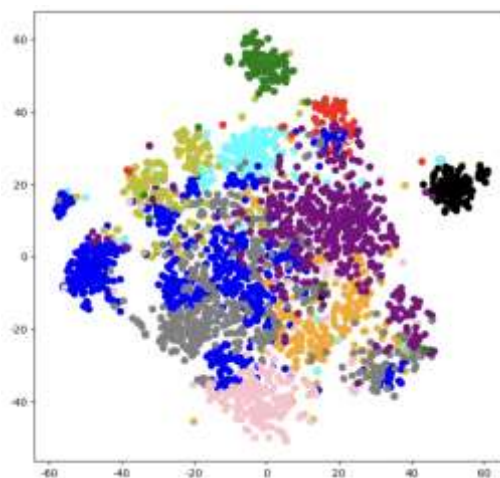
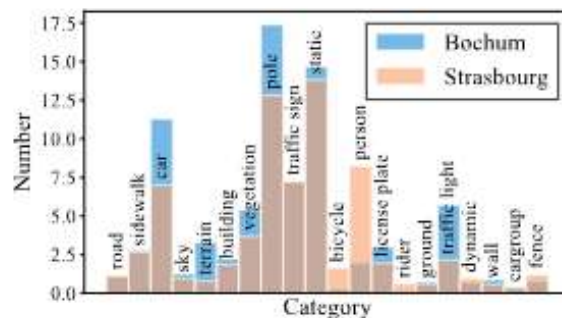
$\mathbf{W}$ 的闭式解则是 $\mathbf{S}_w^{-1} \mathbf{S}_b$ 的 $N-1$ 个最大广义特征值所对应的特征向量组成的矩阵。

□ 多分类LDA将样本投影到 $N-1$ 维空间， $N-1$ 通常远小于数据原有的属性数，因此LDA也被视为一种经典的**监督降维技术**

□ 线性判别分析LDA——多分类任务

■ LDA监督降维技术举例——车辆语义分割

✓ 把一个高维的数据投影到二维空间，然后分类



- 实际中LDA主要用于降维
- 一般来说，如果数据是有类别标签的，那么优先选择LDA去尝试降维，降维到 $N-1$ 维， N 为类别数；
- 当然也可以使用PCA做很小幅度的降维去消去噪声，然后再使用LDA降维。
- 如果没有类别标签，那么肯定PCA是最先考虑的一个选择。

第3章 线性模型

3.1 基本形式

3.2 线性回归

3.3 对数几率回归

3.4 线性判别分析

➡ 3.5 多分类学习

3.6 类别不平衡问题

■ 多分类学习方法

- ✓ 二分类学习方法推广到多类
- ✓ 利用二分类学习器解决多分类问题 (常用)
 - 对问题进行拆分, 为拆出的每个二分类任务训练一个分类器
 - 对于每个分类器的预测结果进行集成以获得最终的多分类结果

■ 拆分策略

- ✓ 一对一 (One vs. One, OvO)
- ✓ 一对其余 (One vs. Rest, OvR)
- ✓ 多对多 (Many vs. Many, MvM)

■ OvO

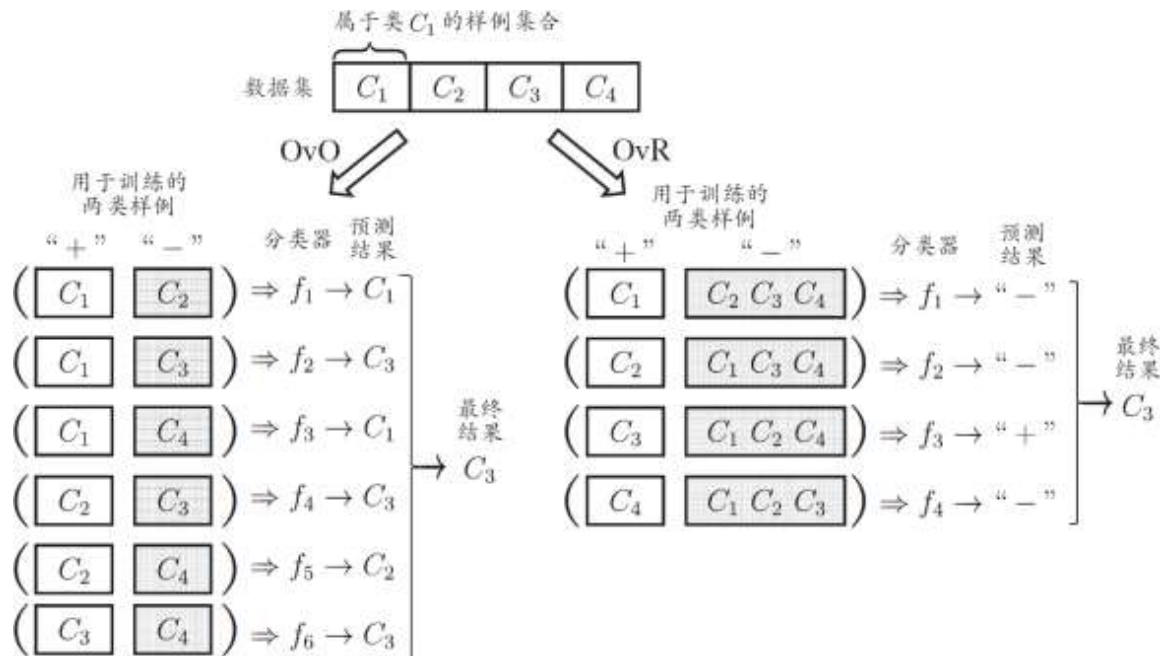
✓ 给定数据集: $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)\}, y_i \in \{C_1, C_2, \dots, C_N\}$

✓ 拆分阶段

- N个类别两两配对
 - 产生 $N(N-1)/2$ 个二类任务
- 各个二类任务学习分类器
 - $N(N-1)/2$ 个二类分类器

✓ 测试阶段

- 新样本提交给所有分类器预测
 - $N(N-1)/2$ 个分类结果
- 投票产生最终分类结果
 - 被预测最多的类别为最终类别



■ OvR

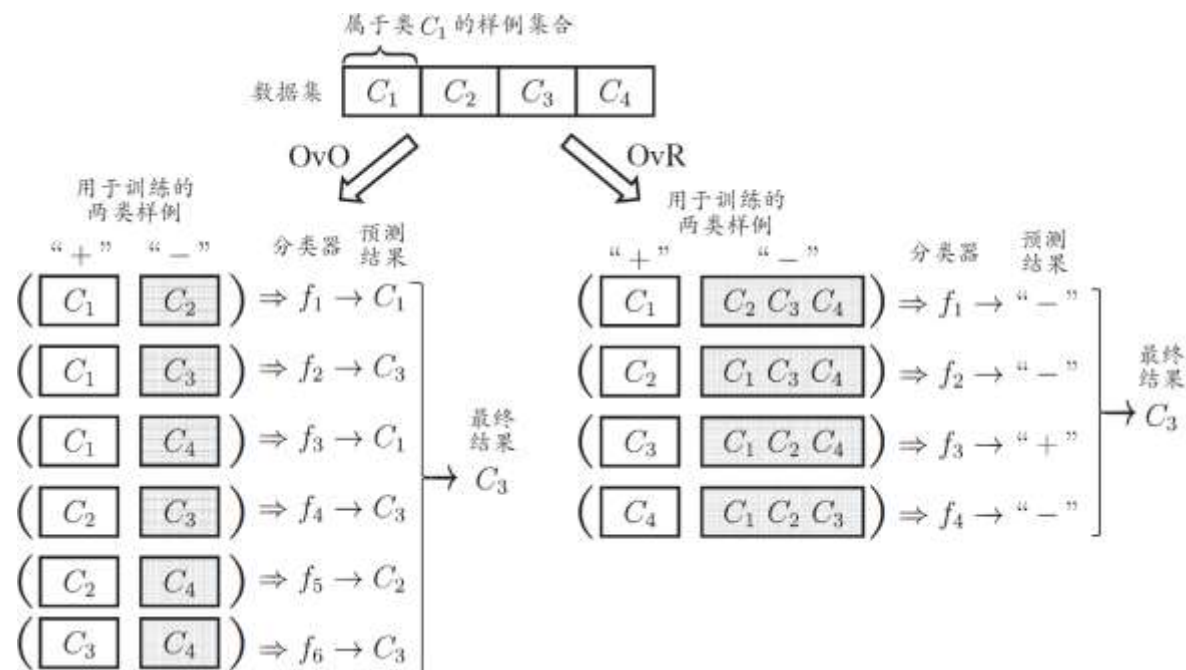
✓ 给定数据集: $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)\}$, $y_i \in \{C_1, C_2, \dots, C_N\}$

✓ 任务拆分

- 某一类作为正例，其他反例
 - N 个二类任务
- 各个二类任务学习分类器
 - N 个二类分类器

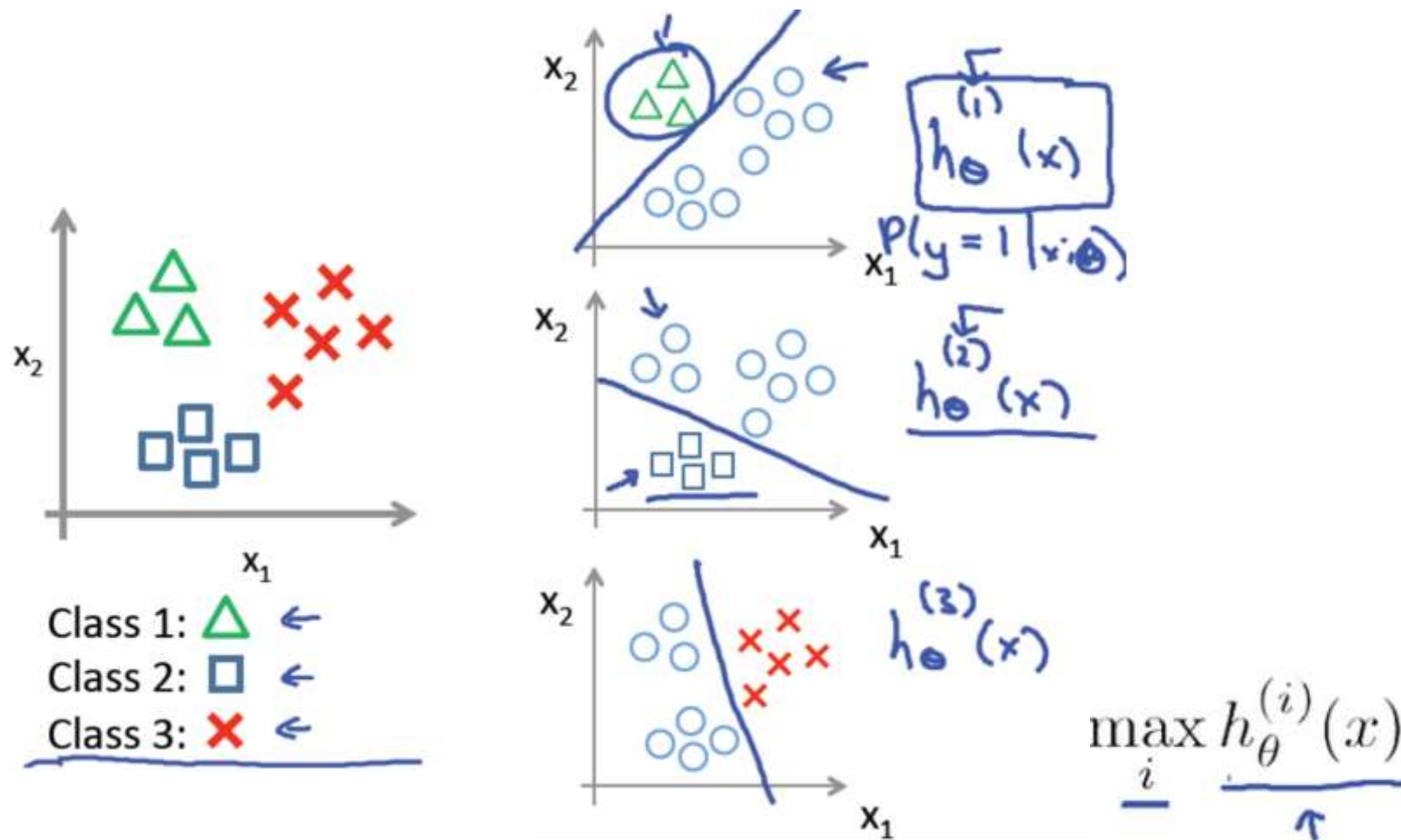
✓ 测试阶段

- 新样本提交给所有分类器预测
 - N 个分类结果
- 比较各分类器**预测置信度**
 - 置信度最大类别作为最终类别



■ 两种策略比较

- ✓ 多个分类器预测为正类，选择置信度最大者



■ 两种策略比较

OvO

- ✓ 训练 $N(N-1)/2$ 个分类器，存储开销和测试时间大
- ✓ 训练只用两个类的样例，训练时间短

OvR

- ✓ 训练 N 个分类器，存储开销和测试时间小
- ✓ 训练用到全部训练样例，训练时间长

预测性能取决于具体数据分布，多数情况下两者差不多

第3章 线性模型

3.1 基本形式

3.2 线性回归

3.3 对数几率回归

3.4 线性判别分析

3.5 多分类学习



3.6 类别不平衡问题

■ 类别不平衡 (class imbalance)

- ✓ 介绍的分类学习方法都有一个共同的基本假设，即不同类别的训练样例数目相当。
如果不同类别的训练样例数目稍有差别，通常影响不大，但若差别很大,则会对学习过程造成困扰
- ✓ 例如有**998**个反例，但正例只有**2**个，那么学习方法只需返回一个永远将新样本预测为反例的学习器，就能达到99.8%的精度；
- ✓ 然而这样的学习器往往没有价值，因为它不能预测出任何正例。

□ 类别不平衡问题

■ 类别不平衡 (class imbalance)

- ✓ 不同类别训练样例数相差很大情况 (正类为小类)
- ✓ 把y与阈值比较, 比如 $y > 0.5$ 判别为正, y代表正例可能性

类别平衡正例预测

若 $\frac{y}{1-y} > 1$ 则 预测为正例



类别不平衡正例预测

若 $\frac{y}{1-y} > \frac{m^+}{m^-}$ 则 预测为正例

• 再缩放

- 欠采样 (undersampling)
 - 去除一些反例使正反例数目接近 (EasyEnsemble [Liu et al.,2009])
- 过采样 (oversampling)
 - 增加一些正例使正反例数目接近 (SMOTE [Chawla et al.2002])
- 阈值移动 (threshold-moving)

■ 各任务下（回归、分类）各个模型优化的目标

- 最小二乘法：最小化均方误差
- 对数几率回归：最大化样本分布似然
- 线性判别分析：投影空间内最小（大）化类内（间）散度

■ 参数的优化方法

- 最小二乘法：线性代数
- 对数几率回归：凸优化梯度下降、牛顿法
- 线性判别分析：矩阵论、广义瑞利商

■ 线性回归

- 最小二乘法（最小化均方误差）

■ 二分类任务

- 对数几率回归
 - 单位阶跃函数、对数几率函数、极大似然法
- 线性判别分析
 - 最大化广义瑞利商

■ 多分类学习

- 一对一
- 一对其余
- 多对多
 - 纠错输出码

■ 类别不平衡问题

- 基本策略：欠采样、过采样、再缩放



重庆大学
CHONGQING UNIVERSITY

THANKS
祝学习快乐!